Mikhail Razakov

Compiler Engineer · LLVM · Program Analysis

misha13022008@gmail.com
t.me/Micodiy
github.com/Misha1302

Compiler engineer spanning LLVM middle-end transformations, program/static analysis, and compiler infrastructure. At MCST, I implemented an LLVM 22 LICM pass and a conservative interprocedural global-state promotion pass; at ISP RAS, I contribute to SharpChecker, an industrial C#/.NET static-analysis platform. Independent work covers Roslyn fixed-point data flow, SSA/x86-64 code generation and register allocation, and deterministic language composition.

EDUCATION

HSE University — Software Engineering, 2026–2030 Moscow / remote

EXPERIENCE

2026 — present

ISP RAS — Static Analysis Engineer

SharpChecker · C#/.NET · industrial static analysis

- Contribute to SharpChecker, ISP RAS's industrial static-analysis platform for C#/.NET.

July — August 2026

MCST — Compiler Engineering Intern

LLVM 22 · C++23 · middle-end / interprocedural analysis

- Implemented an LLVM LICM pass with loop analysis, side-effect/speculative-safety checks, and hoisting of loop-invariant instructions to preheaders.
- Built a conservative interprocedural LLVM pass for localizing induction-like integer globals: APInt affine evolution, transitive call effects, dominance/must-execute legality, and explicit synchronization around known calls; unsupported cases fail closed.

SELECTED RESEARCH & ENGINEERING

LLVM Interprocedural Global IV Optimization

APInt affine evolution + transitive call-effect analysis + fail-closed legality; 29 transform/reject regressions use LLVM Verifier, idempotence, and executable before/after checks.

DerefAfterNullAnalyzer

Roslyn CFG worklist/fixed-point null analysis with edge-sensitive branch states, joins, loops/back edges, and assignment/ref/out invalidation.

x86-64 Codegen & Register Allocation Lab

SSA-like IR → liveness/interference → deterministic register allocation → phi lowering → SysV x86-64; independent RA verifier + differential execution vs a reference interpreter.

UniversalToolchain

Typed contracts compile dependencies/providers/order/routes into an immutable LanguagePlan; exact runtime binding avoids semantic replanning, backed by a 1,324-test exact manifest.

SKILLS

Compilers / LLVM	C++23, LLVM 22, LLVM IR, optimization passes, interprocedural analysis, CFG/SSA, dominators, loop analysis, APInt, transformation legality, LLVM Verifier, CMake, Linux
Program / Static Analysis	C#, .NET, Roslyn ControlFlowGraph, worklist/fixed-point data flow, edge-sensitive states, branch refinement, joins, loops/back edges, conservative analysis
Backend / Code Generation	Rust, SSA-like IR, liveness, interference, deterministic register allocation, phi lowering, SysV x86-64, differential execution
Compiler Infrastructure & Verification	typed artifact/component contracts, deterministic planning/pass ordering, provider/capability resolution, exact runtime binding, idempotence and differential checks

RECOGNITION

LangDev'26 — accepted speaker	Accepted technical talk on UniversalToolchain's architecture; LangDev'26 takes place October 8–9, 2026 in Málaga.
Compiler / engineering competitions	Grand Prize at the Baltic Science and Engineering Competition; MEPhI Junior overall winner, 96/100 for UniversalToolchain.